MPD Suite - Third Party Exports

MATLAB parser

Brief overview of the exemplary MATLAB functions provided by OMICRON

OMICRON Technical Support | support@omicronenergy.com

Content

1 Instruction 3
2 PD Parser 3
3 Phase parser 3
4 Line trigger timestamps parser 3
5 AC parser 3
6 Additional tools 4

© OMICRON 2023

1 Instruction

Parsers for the AC, PD, Phase, and line trigger timestamps data have been created for MATLAB. The individual parsers are briefly described below.

2 PD Parser

PD data is stored in sequential pairs of magnitudes, and timestamps. The magnitudes are encoded as 32-bit floating point numeric values, whereas the timestamps are 64-bit ones. Depending on the measurement mode, the magnitude is represented in either charge (C) or voltages (V). The `importPDData.m` script can be used to return a pair of 1D arrays containing the timestamps (relative to the dataset's playback start time), and magnitudes. An example of running the function is given below:

```
[time_stamps, magnitudes] = importPDData("c:\\20230329_12h19m23s", "MPD 800
2.1.1-20210607_16h35m32s592ms")
```

3 Phase parser

In addition to the magnitudes and timestamps stored in the `.PD` file, the phase position of each PD event with respect to the AC cycle is stored in a `.PH` file. The phase positions are stored as 64-bit floating point numeric values. Additionally, the phase data has the same order as the PD data, making cross-referencing each PD event, its timestamp, and its phase position straightforward.
N.B.: The phase information is written in a range of 0 to 1, with 0 corresponding to 0°, and 1 corresponding to 360°. The `importPHData.m` script can be used to return a 1D array containing the data in the `.PD` file. An example is given below:

```
phase_positions = importPHData("c:\\20230329_12h19m23s", "MPD 800 2.1.1-
20210607_16h35m32s592ms")
```

4 Line trigger timestamps parser

The `importTMData.m` script can be used to parse the firing times of the line trigger. The timestamps will correspond to the 0° phase position observed when using the line trigger as a synchronization source and are relative to the start-time of the dataset playback. An example is given below

```
line_trigger_time_stamps = importTMData("c:\\20230329_12h19m23s", "MPD 800
2.1.1-20210607_16h35m32s592ms")
```

5 AC parser

For AC measurements, the MPD Suite computes a set of Fourier coefficients every 300 ms that represents the AC signal at the defined measurement point. This data is exported to `.AC` files, as blocks containing each Fourier coefficient set produced by the MPD Suite, making it possible to reconstruct the continuous signal. The format of the blocks is as follows:

- Timestamp as a 64-bit floating point numeric value,
- The dominant AC frequency as a 64-bit floating point numeric value,
- The DC component as a 64-bit floating point numeric value,
- The number of Fourier coefficients as an 8-bit unsigned integer,
- n-number of Fourier coefficient pairs as 64-bit floating point numeric values
    - Stored as *coefficient #1 real*, then *coefficient #2 imaginary*, until *coefficient #n real*, *coefficient #n imaginary*...

The `importACData.m` script can be used to import `.AC` files. An example of using the attached script is given below:

```
[ac_coefs, instantaneous_voltage_at, voltages_in_range]=importACData(folderPath,
fileName)
```

© OMICRON 2023

To get the instantaneous voltage at a given timestamp (relative the time of the beginning of the dataset playback), use the following function:

```
instantaneous_voltage_at(time_stamp, ac_coefs)
```

To get voltages across a time range, use the following function:

```
time_range = t_start:t:_step:t_end
voltages_in_range(time_range, ac_coefs)
```

For example, to get a vector with AC "samples" between time 0 and time 10s, which one sample every 10ms, you can use:

```
ac_samples = voltages_in_range(0:10e-3:10, ac_coefs)
```

6 Additional tools

The `histogram.m` and `standardColormap.m` scripts allow you to construct PRPD images from the exported data. You use it as follows:

```
colormap(standardColormap());
[histogram_counts, histogram_colors, histogram_img] = histogram(time_vector, phase_vector, pdevents_vector, pdMin, pdMax, bipolar, logarithmic, xDimension, yDimension)
```

where `time_vector` and `pdevents_vector` are the vectors returned by the importPDData script, `phase_vector` is the vector returned by the importPHData script, `pdMin` and `pdMax` are the lower and upper diagram boundary of the PRPD to be created, `bipolar` creates a bipolar PRPD when true, `logarithmic` enables logarithmic scaling of the PRPD when true, and `xDimension` and `yDimension` determine the width and height of the returned histogram, respectively. `histogram_counts` is a 2-dimensional matrix containing the PD event counts at each (x, y) position, while `histogram_colors` is a 2-dimensional matrix containing the index pf appropriate colors for the (x, y) histogram positions. For convenience, `histogram_img` is a flipped version of `histogram_colors` that can be passed directly to the image function to show the PRPD:

```
image(histogram_img)
```

© OMICRON 2023

OMICRON is an international company serving the electrical power industry with innovative testing and diagnostic solutions. The application of OMICRON products allows users to assess the condition of the primary and secondary equipment on their systems with complete confidence. Services offered in the area of consulting, commissioning, testing, diagnosis and training make the product range complete.

Customers in more than 140 countries rely on the company's ability to supply leading edge technology of excellent quality. Service centers on all continents provide a broad case of knowledge and extraordinary customer support. All of this together with our strong network of sales partners is what has made our company a market leader in the electrical power industry.

For more information, additional literature, and detailed contact information of our worldwide offices please visit our website.

www.omicronenergy.com